

Parallel Optimized Particle Swarm Optimization: An Efficient Parallel Approach for Neural Network Training

Konstantinos G. Barkas*, Ioannis G. Tsoulos, Vasileios Charilogis, and Glykeria Kyrou

Department of Informatics and Telecommunications, University of Ioannina, 47150 Kostaki Artas, Greece

* Author to whom correspondence should be addressed.

Abstract

Global optimization has become a cornerstone of modern machine learning, particularly for the challenging task of training high-dimensional neural network architectures. Despite the prevalence of established metaheuristics, traditional approaches frequently struggle with the irregular and multimodal error surfaces inherent in complex training landscapes. This paper proposes the Parallel Optimized Particle Swarm Optimization (OPSO), a multi-population parallel framework specifically engineered to enhance the training efficiency and predictive accuracy of neural networks across classification and regression tasks. OPSO integrates several advanced mechanisms: a simulated annealing-based acceptance criterion to escape local minima, adaptive inertia weights that dynamically respond to stagnation by injecting momentum, and a hybrid Broyden–Fletcher–Goldfarb–Shanno (BFGS) local search phase for fine-grained convergence. By partitioning the swarm into interacting sub-populations and leveraging modern multi-core architectures, OPSO significantly reduces training time while achieving superior generalization compared to classical PSO and state-of-the-art methods. Experimental evaluations demonstrate that OPSO consistently converges to more robust solutions, outperforming baseline models on demanding and irregular error surfaces. These results underscore OPSO as a highly efficient and scalable framework for optimizing neural network parameters in complex search spaces.

Keywords: Parallel Metaheuristics; Particle Swarm Optimization (OPSO); Neural Network Training; Global Optimization; Hybrid Local Search; Adaptive Inertia; Simulated Annealing

1. Introduction

The training of Artificial Neural Networks (ANNs) represents one of the most critical and challenging tasks in modern machine learning and computational intelligence [1,2]. From a mathematical perspective, training an ANN is formulated as a large-scale global optimization problem [3], where the objective is to minimize an error function by adjusting the network's weights and biases. The global optimization problem is defined as:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in S} f(\mathbf{x}) \quad (1)$$

where the search space S is defined as the high-dimensional hyper-rectangle:

$$S = [a_1, b_1] \otimes [a_2, b_2] \otimes \cdots \otimes [a_n, b_n]$$

In the context of neural network training, the objective function $f(\mathbf{x})$ represents the loss function (e.g., Mean Squared Error or Cross-Entropy), and the vector $\mathbf{x} \in \mathbb{R}^n$ contains the network's parameters. This optimization landscape is highly non-linear, non-convex,

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the

[Creative Commons Attribution \(CC BY\)](#) license.

and multi-modal, containing an immense number of local minima [4]. Apart from neural network training, this mathematical formulation finds critical applications in a variety of real-world problems, such as physics [6,7], chemistry [8], economics [9], and medicine [10,11]. Historically, optimization methods are divided into two main categories: deterministic and stochastic. Deterministic approaches, such as Interval methods [12,13] or gradient-based algorithms (e.g., Backpropagation, Stochastic Gradient Descent, Adam), rely on mathematical proofs or gradient derivatives to navigate the search space [15]. However, in high-dimensional and non-convex neural network landscapes, gradient-based methods are heavily prone to getting trapped in saddle points or sub-optimal local minima and are highly sensitive to initial weight configurations [4,14,15]. To overcome these limitations, stochastic and metaheuristic methods have been widely adopted. Guided by randomness, these methods do not guarantee finding the absolute global minimum in polynomial time, but they consistently locate high-quality solutions with remarkable implementation simplicity [16].

Researchers have proposed numerous stochastic algorithms, including Controlled Random Search [17], Simulated Annealing [18,19], Differential Evolution [20], Particle Swarm Optimization (PSO) [21,22], Ant Colony Optimization [23], and Genetic Algorithms [24]. Furthermore, modern computational paradigms have successfully ported these algorithms to multi-core and GPU architectures to accelerate high-dimensional optimization [25,26], with comprehensive surveys available in recent literature [27,28].

Among these metaheuristics, Particle Swarm Optimization (PSO) has emerged as a highly efficient population-based algorithm inspired by the social behavior of bird flocking [29]. In PSO, a population of candidate solutions, called particles, explores the search space S . Each particle i is characterized by its current position x_i and its velocity v_i . The dynamic movement of each particle in a D -dimensional space is governed by updating its velocity and position at each iteration step [22,35] according to the following formulation:

$$v_{i,d} = \omega \cdot v_{i,d} + c_1 \cdot r_1 \cdot (\mathbf{p}_{\text{best},i,d} - x_{i,d}) + c_2 \cdot r_2 \cdot (\mathbf{g}_{\text{best},d} - x_{i,d}) \quad (2)$$

$$x_{i,d} = x_{i,d} + v_{i,d} \quad (3)$$

where ω is the inertia weight introduced to control the impact of the previous velocity [36], c_1 and c_2 are the cognitive and social acceleration coefficients, and $r_1, r_2 \sim U(0,1)$ are uniformly distributed random numbers. The vector $\mathbf{p}_{\text{best},i}$ denotes the personal best position discovered by particle i , while $\mathbf{g}_{\text{best}}$ represents the global best position found by the entire swarm. Despite its success, classical PSO can suffer from premature convergence and high computational costs when optimizing complex neural network architectures, falling into local minima during the exploitation phase [21,22]. Many researchers focus on dynamically adjusting the inertia weight ω [30,31] or integrating hybrid and parallel implementation techniques [32,33] to tackle complex numerical optimization problems. However, limited attention has been given to the simultaneous enhancement of scalability, multi-population parallel evolution, and adaptive convergence control in large-scale optimization environments. To address these challenges, this paper introduces the Parallel Optimized Particle Swarm Optimization (Parallel OPSO) method, specifically engineered to enhance the training efficiency and predictive accuracy of neural networks. The proposed algorithm is a multi-population parallel framework that partitions the swarm into interacting sub-populations, leveraging modern multi-core architectures to significantly reduce execution time.

The novelty of Parallel OPSO lies in the integration of several advanced mechanisms within a unified parallel framework:

- Simulated Annealing Acceptance Criterion: A stochastic acceptance criterion is integrated to allow particles to occasionally accept worse positions, enabling the swarm to escape stubborn local minima [18,19].
- Adaptive Inertia Weight and Momentum: The inertia weight ω dynamically responds to swarm stagnation by injecting momentum, balancing wide-range exploration and local exploitation based on the algorithm's real-time performance [31].
- Hybrid Local Search Phase: Incorporates a deterministic BFGS quasi-Newton method to refine promising candidate solutions and significantly enhance convergence accuracy toward the global optimum [34].

The main contributions of this work are summarized as follows:

- A parallel cooperative extension of the PSO algorithm, where the swarm is partitioned into multiple sub-populations evolving simultaneously on multi-core architectures using the OpenMP framework to enhance exploration in large-scale search spaces.
- A dynamic propagation mechanism that enables structured and controlled information exchange among the independent subpopulations, improving convergence speed while maintaining overall population diversity.
- The incorporation of adaptive inertia weight ω based on the framework introduced in [31], which dynamically responds to swarm stagnation by injecting momentum, effectively balancing global exploration and local exploitation based on the algorithm's real-time performance.
- The integration of a Simulated Annealing-based stochastic acceptance criterion [18,19], which allows particles to occasionally accept worse positions, providing a robust mechanism to escape stubborn local minima.
- The incorporation of a hybrid local search phase utilizing the deterministic BFGS quasi-Newton method [34] to significantly improve convergence accuracy and speed up the localization of the exact global optimum.
- A comprehensive experimental evaluation spanning both large-scale global optimization benchmark problems and demanding neural network training tasks for classification and regression, demonstrating that Parallel OPSO consistently converges to more robust solutions compared to state-of-the-art algorithms.

The remainder of this article is organized as follows: Section 2 outlines the foundational background of classical PSO alongside the detailed mathematical formulation of the proposed Parallel OPSO; Section 3 presents the experimental setup, benchmark functions, and comparative performance analysis; and finally, Section 4 summarizes our key findings, provides concluding remarks, and outlines directions for future research.

2. Materials and Methods

This section initiates with the main aspects of the original PSO method and continues by presenting in detail the main parts of the proposed method.

2.1. Classical Particle Swarm Optimization (PSO)

The classical Particle Swarm Optimization (PSO) algorithm, originally introduced by Kennedy and Eberhart [35], is a population-based stochastic optimization technique inspired by the social behavior of bird flocking or fish schooling. The algorithm explores an n -dimensional continuous search space $\mathcal{S} \subset \mathbb{R}^n$ using a population of m interacting agents called particles. Each particle i ($i \in \{1, 2, \dots, m\}$) is characterized by its dynamic state parameters, consisting of an n -dimensional position vector $\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{in}]^T$ and

a corresponding velocity vector $\mathbf{v}_i = [v_{i1}, v_{i2}, \dots, v_{in}]^T$. During the optimization process, each particle maintains a memory of its own historical personal best position discovered so far, denoted as $\mathbf{p}_i = [p_{i1}, p_{i2}, \dots, p_{in}]^T$. Concurrently, the swarm tracks the absolute global best position achieved by any particle in the entire population, defined as $\mathbf{p}_{gbest} = \arg \min_{i \in \{1, \dots, m\}} f(\mathbf{p}_i)$, where $f(\cdot)$ represents the objective function under minimization. To balance global exploration and localized exploitation, a trade-off extensively evaluated in recent metaheuristic literature [2,3], Shi and Eberhart [36] introduced the inertia weight factor ω . At each discrete evolutionary epoch t , the dynamic trajectories of the particles are updated according to the following stochastic kinematic equations:

$$\mathbf{v}_i(t+1) = \omega(t) \cdot \mathbf{v}_i(t) + c_1 \cdot \mathbf{r}_1 \cdot (\mathbf{p}_i(t) - \mathbf{x}_i(t)) + c_2 \cdot \mathbf{r}_2 \cdot (\mathbf{p}_{gbest}(t) - \mathbf{x}_i(t)) \quad (4)$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1) \quad (5)$$

where $\omega(t)$ is the inertia weight, c_1 and c_2 denote the cognitive and social acceleration coefficients, respectively, and $\mathbf{r}_1, \mathbf{r}_2 \sim \mathcal{U}(0, 1)$ represent independent uniformly distributed random vectors injected to maintain stochastic diversity. Despite its mathematical simplicity and high convergence speed, the classical formulation is prone to stagnation in complex multimodal landscapes due to rapid loss of population diversity, a challenge frequently highlighted in modern optimization benchmarks [5,21]. The main parts of the original PSO method are listed in Algorithm 1.

Algorithm 1: The classical PSO algorithm.

```

/* Initialization Phase */
1 Set iter  $\leftarrow 0$  (iteration counter)
2 Set the population size  $m$  and maximum iterations allowed  $iter_{max}$ 
3 Initialize randomly the positions of the  $m$  particles  $x_1, x_2, \dots, x_m$ , with
    $x_i \in \mathcal{S} \subset \mathbb{R}^n$ 
4 Initialize randomly the velocities of the  $m$  particles  $v_1, v_2, \dots, v_m$ 
5 for  $i \leftarrow 1$  to  $m$  do
6    $p_i \leftarrow x_i$  // Store best personal position
7 Find  $g = \arg \min_{i \in \{1, \dots, m\}} f(x_i)$  and set  $p_{gbest} \leftarrow x_g$ 
/* Main Evolutionary Loop */
8 while  $iter < iter_{max}$  do
9   for  $i \leftarrow 1$  to  $m$  do
10    Update the velocity  $v_i$  via Eq. (4)
11    Update the position  $x_i$  via Eq. (5)
12    Evaluate the fitness of particle  $i$ ,  $f(x_i)$ 
13    if  $f(x_i) \leq f(p_i)$  then
14       $p_i \leftarrow x_i$ 
15 Find  $g = \arg \min_{i \in \{1, \dots, m\}} f(p_i)$  and set  $p_{gbest} \leftarrow p_g$ 
16 iter  $\leftarrow iter + 1$ 
17 return  $p_{gbest}, f(p_{gbest})$ 

```

2.2. Parallel Optimized Particle Swarm Optimization Algorithm

The distributed framework of the Optimized Particle Swarm Optimization (OPSO) algorithm is engineered by partitioning the global agent population into K distinct subpopulations and it is presented graphically in Figure 1. Each subpopulation operates as an autonomous evolutionary unit allocated to an individual processing element, enabling simultaneous execution across a shared-memory environment [37].

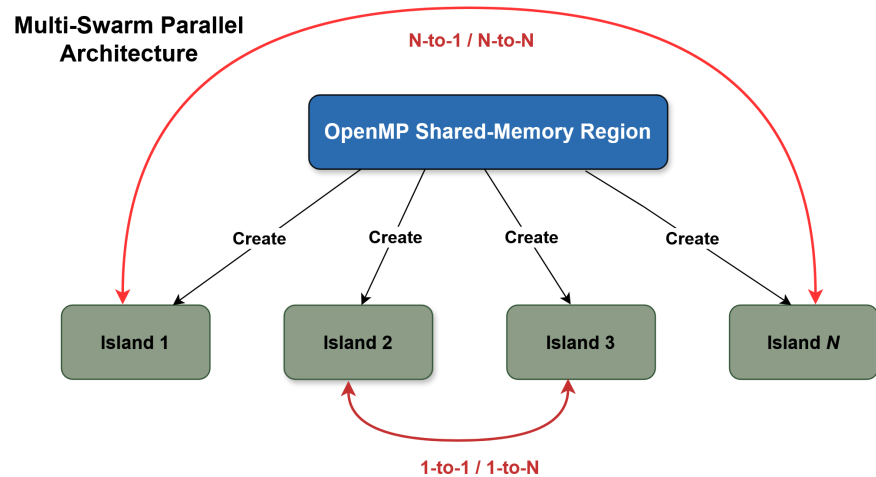


Figure 1. Parallel OPSO framework illustrating thread distribution and flexible inter-island migration topologies.

Before initiating the optimization cycle, the multi-swarm configurations are established, and the global termination strategy (TS) is selected. To govern decentralized convergence behavior and manage interaction dynamics between the concurrently evolving swarms, the architecture introduces a set of control features: a topological propagation method (NM), a generational communication rate (NR), a volumetric quantity of migrated solutions (NP), and a localized termination criterion (TC) [33,38].

The communication frequency across the multi-swarm layout is controlled by the propagation rate (NR), which establishes the exact number of generational steps between consecutive data distributions. Low values for NR enforce tight coupling and frequent interaction, a configuration that can accelerate convergence velocities but risks collapsing population diversity [38,39]. Conversely, high values for NR grant the individual islands extended periods of decoupled execution, fostering independent exploration of unmapped solution regions.

In a similar manner, the volume of shared elite information is dictated by the parameter NP, which defines the number of top-performing vectors transmitted during a migration step. Expanding NP strengthens the collective exploitation capability by injecting high-quality geometric coordinates into target swarms; however, excessive data injection can induce premature convergence by steering all independent groups toward identical local optima [38–40]. Within this framework, the specific bounds for NR and NP are derived through systematic empirical tuning to maintain a healthy balance between localized search refinement and global space exploration, with the precise values detailed in the Experimental Results section.

During every generational step, the system monitors each subpopulation to determine if it has met its localized termination criterion (TC), which is evaluated independently via an advanced convergence heuristic known as the Similarity check[41]. The total number

of stagnated units is then cross-referenced with a macro-level decision threshold, denoted as Q . This threshold mathematically operationalizes the chosen system-wide termination strategy (TS) to govern the macro-level optimization loop[38,41]:

$$Q = \begin{cases} 1, & \text{if } TS = ANY \\ \lfloor K/2 \rfloor + 1, & \text{if } TS = MAJORITY \\ K, & \text{if } TS = ALL \end{cases} \quad (6)$$

Following the calculation of the threshold metrics, the subpopulations proceed with their parallel evolutionary operations, executing localized kinematics updates and simulated annealing leader adjustments. At generational intervals matching the propagation rate (NR), the system triggers the designated migration topology (NM) to distribute NP elite candidate vectors across the multi-threaded network[33,38,40].

The iteration counter increments, and the status of the threshold criterion Q is evaluated. If the conditions for global termination are not fulfilled, the concurrent optimization sequence continues. Upon satisfying the macro-level stopping condition, the framework invokes a memetic refinement sequence, executing a rigorous local search directly around the global best solution g_{best} to secure high-precision mathematical accuracy before returning the final optimized result [42].

From an architectural standpoint, mapping each subpopulation to an isolated thread or processing core facilitates true concurrent execution. Synchronization barriers are strictly confined to the scheduled propagation phases, allowing the swarms to evolve independently during intermediate generations and significantly minimizing memory access latencies [33]. Resource utilization is optimized through the selection of the global termination strategies (ANY, MAJORITY, ALL), which prevent thread starvation and idle processing states by eliminating the requirement for all processing elements to terminate at the same time. The ANY strategy, in particular, permits rapid continuation by halting the global loop the moment early convergence is detected on any single thread via the Similarity metric [41]. Furthermore, the computational footprint of the migration phase remains small, as it restricts communication to a limited set of elite solution coordinates. This brief coordination phase introduces negligible latency relative to the overall objective function evaluation costs, and it significantly enhances optimization performance by enabling effective information sharing across the parallel architecture [33,38].

2.3. Inter-Swarm Communication Protocol and Migration Topologies

Within the cooperative execution phase of the parallel framework, the communication layer facilitates the distribution of high-quality parameter configurations among the concurrent islands. When the scheduled migration interval conditions are satisfied, the elite solution vectors identified by a specific subpopulation are transmitted across the shared-memory space to overwrite the lowest-performing coordinates of the designated target swarms. The foundational objective of this information routing scheme is to optimize cross-island exploration, ensuring that promising topological discoveries are rapidly assimilated into the broader optimization landscape to enhance overall convergence efficiency [33,38].

Unlike conventional distributed metaheuristics that rely on static and rigid structural layouts, such as ring, star, or fully-mesh migration topologies, the proposed framework introduces a localized, dynamically configurable migration model tailored for high-dimensional execution spaces. Conventional geometric topologies impose strict synchronization locks

and significant thread contention penalties as the execution scale grows, which frequently induces severe memory access latencies within shared-memory architectures [33,38]. To avoid structural complexity and eliminate excessive communication overhead, the parallel architecture relies on a versatile routing protocol operating across four discrete configuration states:

- One-to-One (1to1): A single source subpopulation, selected uniformly at random, migrates its historical elite parameter states exclusively to a single target subpopulation, which is also designated through a random uniform distribution.
- One-to-All (1toN): A randomly isolated subpopulation serves as a central broadcaster, transmitting its optimal geometric updates to refresh the search vectors of all other $K - 1$ active swarms simultaneously.
- All-to-One (Nto1): Every active computational swarm routes its current top-performing vectors toward a single, randomly targeted subpopulation, consolidating global directional data into a localized spatial window to catalyze structural exploitation.
- All-to-All (NtoN): A dense execution mode where every individual subpopulation synchronizes and exchanges its finest candidate solutions with the historical memory matrix of every other swarm in the network.

By adjusting the combination of these topological routing strategies (NM) alongside the volumetric payload parameter (NP), the OPSO framework maintains a fluid balance between regional exploitation and global population diversity [33,38]. This dynamic coordination mitigates structural stagnation risks while preserving the low coordination footprint required for efficient execution on multi-core compute clusters.

2.4. Adaptive Inertia weight mechanism

To achieve a self-regulating balance between global exploration and localized exploitation, each autonomous subpopulation discards static or strictly time-dependent parameter scaling. Instead, an Adaptive Inertia Weight Mechanism is implemented [30], adapting the phenotypic stagnation-tracking framework introduced in the Ideal Particle Swarm Optimization (IPSO) architecture [31]. This framework evaluates the state of the swarm by monitoring the phenotypic convergence rate across consecutive generations. When the global fitness gradient flattens, indicating that the multi-swarm trajectory is confined within a local attractor, the inertia weight parameter is dynamically adjusted to modulate particle momentum.

Operationally, the system calculates the absolute differential progress of the collective fitness landscape at every iteration. If the fitness deviation between sequential states falls below a predefined threshold, a stagnation counter increments. By calculating the ratio of cumulative stagnant states against the elapsed generational counter, a dynamic constraint coefficient is derived. Consequently, when the subpopulation actively discovers novel structural minima, the velocity profile maintains a high exploratory momentum. Conversely, when severe multi-island stagnation is detected, the inertia scales down to its minimum operating boundary, dampening particle velocities to focus computational resources strictly on high-precision local exploitation[30,31].

2.4.1. Mathematical Formulation

The evolutionary state monitoring is governed by evaluating the global fitness volume change at each generation t , tracking structural stagnation vectors as formalized in the IPSO framework [31]:

$$\delta(t) = \left| \sum_{i=1}^m f_i(t) - \sum_{i=1}^m f_i(t-1) \right| \quad (7)$$

where m denotes the number of particles within the subpopulation, and $f_i(t)$ represents the objective cost value of the i -th particle at generation t . The localized stagnation state indicator $\zeta(t)$ is defined as:

$$\zeta(t) = \begin{cases} 1, & \text{if } \delta(t) = 0 \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

Using the real-time feedback from $\zeta(t)$, the cumulative stagnation tracking matrix $S_\delta(t)$ and its corresponding density ratio $C_\delta(t)$ are computed dynamically:

$$S_\delta(t) = \sum_{k=1}^t \zeta(k) \quad (9)$$

$$C_\delta(t) = \frac{S_\delta(t)}{t} \quad (10)$$

The adaptive inertia weight $\omega(t)$ supplied to the velocity kinematic updating core is formally governed by the bound-constrained formulation[31], building upon the dynamic inertia weight control concepts introduced in [36]:

$$\omega(t) = \max(\omega_{\min}, \min(\omega_{\max}, \omega_{\max} - C_\delta(t) \cdot (\omega_{\max} - \omega_{\min}))) \quad (11)$$

where $\omega_{\max}$ and $\omega_{\min}$ represent the predefined boundary coefficients governing the dynamic momentum spectrum.

2.5. Adaptive Simulated Annealing Acceptance Criterion

To balance the exploitation of local optima with the necessity of maintaining exploratory pressure, a stochastic acceptance mechanism is integrated into the leader update phase of each island. Traditional PSO variants typically employ a greedy selection rule, which often leads to rapid convergence toward suboptimal regions in non-convex, high-dimensional landscapes [22,29]. To mitigate this, our approach implements an Adaptive Simulated Annealing Acceptance Criterion, allowing the sub-population leader ($\mathbf{P}_{\text{best},k}$) to occasionally accept inferior solutions based on a temperature-dependent probability inspired by the classic Simulated Annealing framework [6,18,19].

Let $\Delta = f(\mathbf{X}_{\text{best},k}) - f(\mathbf{P}_{\text{best},k})$ represent the fitness difference between the current best particle within sub-cluster S_k and the recorded sub-cluster leader. The probability P_{acc} of accepting a potentially worse configuration is defined as:

$$P_{\text{acc}} = \begin{cases} 1.0, & \text{if } \Delta < 0 \\ \exp\left(-\frac{\Delta}{T \cdot \Delta_k + \epsilon}\right), & \text{otherwise} \end{cases} \quad (12)$$

where T represents the current normalized temperature under a linear cooling schedule, calculated as $T = \max(1.0 - \text{progress}, 10^{-4})$ to enforce a minimum thermal floor,

with progress denoting the relative execution progress ($t/t_{\max}$). To ensure robustness across diverse objective function scales, an adaptive scaling factor $\bar{\Delta}_k$ is maintained locally for each sub-cluster k , updating incrementally via an Exponential Moving Average (EMA) as $\bar{\Delta}_k \leftarrow 0.9 \cdot \bar{\Delta}_k + 0.1 \cdot \Delta$ for every positive fitness delta observed. The term $\epsilon = 10^{-9}$ serves as a numerical stability constant to prevent division by zero.

This mechanism facilitates a transition from a high-exploration phase at the onset of the optimization process, where the acceptance of inferior moves enables the algorithm to escape local attractors, to a high-exploitation phase as T approaches zero, effectively narrowing the search towards the global optimum [18,19]. By decoupling the leader update from strict greedy selection, the algorithm exhibits enhanced resilience against the "saddle point" problem in high-dimensional spaces [4], ensuring a more thorough traversal of the search space before settling into a terminal configuration [12,13]. This adaptive control, combined with the parallel island-model paradigm, provides a robust defense against premature stagnation without the prohibitive computational overhead associated with global, non-localized simulated annealing schedules [31,33,39,40].

2.6. Hybrid Local Search Phase

To balance global stochastic exploration with high-precision exploitation, Parallel OPSO incorporates a memetic hybrid local search phase [42] based on the deterministic Broyden–Fletcher–Goldfarb–Shanno (BFGS) quasi-Newton algorithm [34]. During execution, the local search procedure is stochastically triggered on the global best position $\mathbf{g}_{\text{best}}$ with a predefined probability p_{ls} (`local_search_rate`). Additionally, a final deterministic refinement step is mandatorily executed inside the `done()` routine upon meeting the swarm stopping criteria. The BFGS algorithm iteratively refines the target position vector $\mathbf{x}_k$ along a search direction $\mathbf{p}_k = -H_k \nabla f(\mathbf{x}_k)$, where H_k represents an approximate inverse Hessian matrix. At each local step k , H_k is dynamically updated using the spatial difference vectors $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ and the gradient difference vector $\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$ according to:

$$H_{k+1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) H_k (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T \quad (13)$$

where $\rho_k = (\mathbf{y}_k^T \mathbf{s}_k)^{-1}$. This gradient-based coupling guarantees superlinear local convergence characteristics, refining the stochastic candidate solutions of OPSO to exact numerical optima.

2.7. Subpopulation Termination Criterion via Similarity Check

In parallel multi-swarm architectures, determining the optimal execution boundary for each autonomous subpopulation is critical. Halting an island prematurely deprives the global swarm of valuable population diversity and high-quality coordinates during migration phases [39,40]. Conversely, allowing a stagnated subpopulation to execute until the global maximum iteration limit leads to wasted processor cycles and unnecessary energy consumption [33,38]. To resolve this trade-off, each individual island k dynamically monitors its own search trajectory via a localized termination state (TC_k).

To ensure sufficient initial exploration and avoid false-positive termination signals caused by early stochastic fluctuations, a mandatory warm-up period is enforced. Localized convergence diagnostics remain inactive until the generation counter satisfies:

$$t \geq t_{\min}, \quad \text{where } t_{\min} = 50 \quad (14)$$

Upon crossing this threshold, the algorithm tracks the fitness progression of island k at every generation t by calculating the absolute difference between its current and previous local best objective values:

$$\Delta f_k(t) = |f(\mathbf{L}_{\text{best},k}(t)) - f(\mathbf{L}_{\text{best},k}(t-1))| \quad (15)$$

To evaluate whether $\Delta f_k(t)$ reflects genuine mathematical convergence rather than a transient plateau, the framework embeds the Similarity Check heuristic proposed by Tsoulos et al. [41]. Conventional termination metrics often rely on instantaneous point-to-point differences, making them highly sensitive to noise in complex fitness landscapes. In contrast, the Similarity Check evaluates the convergence behavior over a sliding historical window of $W = 25$ consecutive iterations. Specifically, island k is declared locally converged ($TC_k = \text{true}$) when the average objective improvement falls below a predefined tolerance threshold ϵ :

$$\frac{1}{W} \sum_{m=0}^{W-1} \Delta f_k(t-m) < \epsilon \quad (16)$$

From an implementation perspective, each parallel worker thread maintains an isolated heuristic evaluator instance (`threadSimilarity[k]`). This design eliminates thread contention and lock overheads, enabling fully decoupled convergence checks across processing cores [33,37]. The aggregate count of stagnated islands is continuously tracked as:

$$S_{\text{term}} = \sum_{k=1}^K \mathbb{I}(TC_k = \text{true}) \quad (17)$$

where $\mathbb{I}(\cdot)$ denotes the indicator function. When $S_{\text{term}} \geq Q$, the global termination strategy (TS) is triggered [38,41]. Finally, the maximum iteration count t_{max} serves as an absolute safety ceiling ($t \geq t_{\text{max}}$), guaranteeing bounded computational time under all execution conditions.

2.8. Pseudocode and Algorithmic Summary

The complete metaheuristic workflow of the proposed Parallel Optimized Particle Swarm Optimization algorithm, incorporating the parallel migration topologies, adaptive inertia weight control mechanisms, simulated annealing leader acceptance criteria, and hybrid local search refinement, is synthesized in the following pseudocode, presented across three parts to preserve legibility: the initialization phase (Algorithm 2), the main evolutionary loop (Algorithm 3), and the per-island parallel update procedure invoked within it (Algorithm 4):

Algorithm 2: Optimized and Parallel Particle Swarm Optimization (OPSO) — Initialization Phase

Input : Search space bounds $[lb, ub]$, Dimension D , Objective function $f(x)$, Swarm size N_p , Islands K ($S = N_p/K$ particles/island), Max iterations N_{iter} , Acceleration constants c_1, c_2, w_{start} , Flags: `velocity_mode`, `inertia_type`, `prop`, Rates: $p_{ls}, R_{prop}, M_{prop}$

Output: Initialized swarm state, ready for the main evolutionary loop of Algorithm 3

```

/* Initialization Phase */
1 Initialize random seed generators cell_gen[i] for  $i \in \{0, \dots, N_p - 1\}$ ;
2 foreach particle  $i \in \{0, \dots, N_p - 1\}$  do
3    $x_i \sim \mathcal{U}(lb, ub), v_i \leftarrow 0$ ;
4   Evaluate fitness  $f(x_i)$ ;
5    $pbest_i \leftarrow x_i, f(pbest_i) \leftarrow f(x_i)$ ;
6 Initialize shared SA scale  $avg\_delta \leftarrow 0.01, S_\delta \leftarrow 0, prev\_fitness\_sum \leftarrow 0$ ;
7 Partition swarm into  $K$  independent islands  $S_0, \dots, S_{K-1}$  of size  $S$ ;
8 Initialize island best fitness  $bestF2x[k] \leftarrow \infty, \forall k \in \{0, \dots, K - 1\}$ ;
9 foreach island  $k \in \{0, \dots, K - 1\}$  do
10  UpdateIslandLeader(k); // Computes initial leader  $bestSample_k$ 
11  $f(x_{gbest}) \leftarrow \min_k bestF2x[k]$  and update  $x_{gbest}$ ;
12 iter  $\leftarrow 0$ ;
  
```

Algorithm 2 performs the sequential initialization of the swarm, the islands, and the initial leaders. The resulting state is then evolved by the main loop of Algorithm 3, which repeatedly invokes, within its OpenMP parallel region, the per-island update procedure detailed separately in Algorithm 4.

Algorithm 3: Optimized and Parallel Particle Swarm Optimization (OPSO) —
 Main Evolutionary Loop (continues from Algorithm 2)

```

1 while  $iter < N_{iter}$  and not Terminated() do
2    $progress \leftarrow iter / N_{iter};$ 
3    $w_{curr} \sim \mathcal{U}(0.5, 1.0);$ 
4   /* Stagnation Monitor for iPSO Mode */
5   if  $velocity\_mode == "ipso\_vmax"$  and  $inertia\_type == 14$  then
6      $current\_fitness\_sum \leftarrow \sum_{i=0}^{N_p-1} |f(x_i)|;$ 
7     if  $iter == 0$  then
8        $prev\_fitness\_sum \leftarrow current\_fitness\_sum;$ 
9       if  $iter > 0$  and  $|current\_fitness\_sum - prev\_fitness\_sum| == 0$  then
10         $S_\delta \leftarrow S_\delta + 1;$ 
11         $prev\_fitness\_sum \leftarrow current\_fitness\_sum;$ 
12    foreach  $island\ k \in \{0, \dots, K-1\}$  do
13       $bestF2xOld[k] \leftarrow bestF2x[k];$ 
14      /* Parallel Region: OpenMP Execution across Islands */
15      foreach  $island\ k \in \{0, \dots, K-1\}$  in parallel do
16         $ProcessIsland(k);$  // Executes Algorithm 4
17      /* Sequential Region: Simulated Annealing Leader Update */
18      foreach  $island\ k \in \{0, \dots, K-1\}$  do
19         $UpdateIslandLeader(k);$ 
20      /* Inter-Island Migration / Propagation */
21      if  $prop == 1$  and  $K > 1$  and  $iter \pmod{R_{prop}} == 0$  then
22         $Propagate(propagationMethod);$ 
23        foreach  $island\ k \in \{0, \dots, K-1\}$  do
24           $UpdateIslandLeader(k);$ 
25      /* Global Memetic Refinement */
26      if  $\mathcal{U}(0, 1) < p_{Is}$  then
27        pragma omp critical( $gbest\_update$ );
28        begin
29           $x_{Is} \leftarrow LocalSearch(x_{gbest});$ 
30          if  $f(x_{Is}) < f(x_{gbest})$  then
31             $f(x_{gbest}) \leftarrow f(x_{Is}), x_{gbest} \leftarrow x_{Is};$ 
32       $iter \leftarrow iter + 1;$ 
33   $x_{gbest} \leftarrow LocalSearch(x_{gbest});$  // Final local refinement
34  return  $x_{gbest}, f(x_{gbest})$ 

```

Algorithm 4 specifies the computations carried out independently and concurrently for each island during every generation of the main loop, namely the fitness evaluation and personal/global best updates, followed by the particle movement, velocity clamping, and bound enforcement steps.

Algorithm 4: ProcessIsland(k): Parallel Fitness Evaluation and Particle Update for Island k (invoked from the parallel region of Algorithm 3)

Input: Island index k

```

1 start  $\leftarrow k \cdot S$ , end  $\leftarrow \min((k+1) \cdot S, N_p)$ ;
  /* Fitness Evaluation & Personal/Global Best Updates */
2 for  $i \leftarrow start$  to end - 1 do
3   pragma omp critical(statfunmin_eval);
4   begin
5      $fit \leftarrow f(x_i)$ ;
6      $f(x_i) \leftarrow fit$ ;
7     if  $fit < f(pbest_i)$  then
8        $pbest_i \leftarrow x_i$ ,  $f(pbest_i) \leftarrow fit$ ;
9     if  $fit < f(x_{gbest})$  then
10      pragma omp critical(gbest_update);
11      begin
12        if  $fit < f(x_{gbest})$  then
13           $f(x_{gbest}) \leftarrow fit$ ,  $x_{gbest} \leftarrow x_i$ ;
14
15      /* Particle Movement & Velocity Clamping */
16      for  $i \leftarrow start$  to end - 1 do
17        for  $j \leftarrow 0$  to  $D - 1$  do
18           $r_1, r_2 \sim |\mathcal{N}_{[-2,2]}(0,1)|$ ; // Absolute value of truncated normal
19           $old\_x \leftarrow x_{i,j}$ ,  $old\_v \leftarrow v_{i,j}$ ;
20          if  $velocity\_mode \neq "ipso\_vmax"$  then
21             $v_{i,j} \leftarrow w_{curr} \cdot v_{i,j} + c_1 r_1 (pbest_{i,j} - x_{i,j}) + c_2 r_2 (bestSample_{k,j} - x_{i,j})$ ;
22          if  $velocity\_mode == "standard\_vmax"$  then
23             $v_{i,j} \leftarrow \text{Clamp}(v_{i,j}, -13.2, 13.2)$ ;
24          else if  $velocity\_mode == "dynamic\_vmax"$  then
25             $v_{max} \leftarrow 0.2 \cdot (ub_j - lb_j) \cdot (1.0 - 0.5 \cdot progress)$ ;
26             $v_{i,j} \leftarrow \text{Clamp}(v_{i,j}, -v_{max}, v_{max})$ ;
27          else if  $velocity\_mode == "ipso\_vmax"$  then
28             $w_{ipso} \leftarrow \text{CalculateInertia}(inertia\_type, iter, N_{iter}, cell\_gen[k])$ ;
29             $v_{i,j} \leftarrow w_{ipso} \cdot v_{i,j} + 2.0 r_1 (pbest_{i,j} - x_{i,j}) + 1.0 r_2 (bestSample_{k,j} - x_{i,j})$ ;
30             $v_{i,j} \leftarrow v_{i,j} / (ub_j - lb_j)$ ; // Scale velocity by range
31           $x_{i,j} \leftarrow x_{i,j} + v_{i,j}$ ;
32          /* Bound Enforcement */
33          if  $x_{i,j} < lb_j$  or  $x_{i,j} > ub_j$  then
34             $x_{i,j} \leftarrow old\_x$ ;
35             $v_{i,j} \leftarrow old\_v$ ;

```

2.9. Computational Complexity Analysis

The computational complexity of the proposed algorithm is analyzed per iteration, considering the number of particles N , the dimensionality of the search space D , and the number of subpopulations (islands) K . Let $n = N/K$ be the number of particles per island. The total complexity per iteration is dominated by the evaluation of the objective function, the velocity/position updates, the hybrid local search procedure, and the communication overhead. The breakdown is as follows:

- **Objective Function Evaluation:** The primary cost is associated with the evaluation of the objective function. In each iteration, N evaluations are performed. If f_{cost} denotes the computational cost of the objective function $f(\mathbf{x})$, the complexity is $\mathcal{O}(N \cdot f_{cost})$. Due to the OpenMP parallelization, this is distributed across K threads, reducing the execution time to $\mathcal{O}(\frac{N}{K} \cdot f_{cost})$ per thread, assuming uniform load balancing [33,37].
 - **Particle Update Mechanism:** The velocity and position update operations for all N particles across D dimensions require $\mathcal{O}(N \cdot D)$ time. Similar to the objective function, this is parallelized as $\mathcal{O}(\frac{N}{K} \cdot D)$ per subpopulation [33,36].
 - **Hybrid Local Search Phase:** The deterministic BFGS quasi-Newton refinement is stochastically invoked on the global best solution with probability p_{ls} . Let L_{evals} denote the average number of objective function evaluations required by the BFGS gradient calculations and line search per local execution. The computational cost of matrix updates for a D -dimensional search space is $\mathcal{O}(D^2)$. Thus, the amortized cost per iteration for the hybrid refinement phase is $\mathcal{O}(p_{ls} \cdot (L_{evals} \cdot f_{cost} + D^2))$ [34,42].
 - **Inter-Island Propagation:** The migration mechanism involves sorting particles by fitness within subpopulations and transferring them. Sorting n particles takes $\mathcal{O}(n \log n)$. Given that the migration occurs every P iterations (propagation rate), the amortized complexity per iteration for the migration phase is $\mathcal{O}(\frac{K \cdot n \log n}{P})$ [33,39,40].
- Combining the above, the total computational complexity per iteration T_t is expressed as:

$$T_t \approx \mathcal{O}\left(\frac{N}{K}(f_{cost} + D) + p_{ls} \cdot (L_{evals} \cdot f_{cost} + D^2) + \frac{N \log\left(\frac{N}{K}\right)}{P}\right) \quad (18)$$

As P is typically significantly larger than 1 (e.g., $P = 10$) and p_{ls} is set to a moderate rate (e.g., $p_{ls} \leq 0.1$), the propagation and local search overheads remain negligible compared to the objective function evaluations. Consequently, the algorithm exhibits linear scaling with respect to the number of particles and dimensions, while the parallel efficiency is optimized by the factor K , allowing the algorithm to maintain high performance in high-dimensional optimization problems [28,33,38].

3. Results

3.1. Experimental datasets

The datasets used in the conducted experiments were downloaded from the following following online databases:

1. The UCI website, <https://archive.ics.uci.edu/> (accessed on 24 June 2026)[43]
2. The Keel website, <https://sci2s.ugr.es/keel/datasets.php> (accessed on 24 June 2026)[44].

The datasets used in the conducted experiments are:

1. **Appendictis** dataset, which is a medical dataset[45].
2. The **Alcohol** dataset, concerning alcohol consumption, was introduced in [46].
3. **Australian** dataset [47] used in economics.
4. The **Balance** dataset, related to various psychological experiments, was introduced in [48].
5. **Circular** dataset, which represents an artificial dataset with two classes.
6. **Cleveland** dataset, that denotes a medical dataset regarding heart diseases [49,50].
7. **Dermatology** dataset [51], a medical dataset.

8. The **Ecoli** dataset, used for detecting protein localization problems, was introduced in [52].
9. The **Fert** dataset, related to sperm concentration disorders.
10. **Haberman**, a medical dataset related to the presence of breast cancer.
11. **Hayes roth** dataset[53].
12. **Heart** dataset [54], a medical dataset regarding heart diseases.
13. **HeartAttack**, a medical dataset related to heart diseases
14. **HouseVotes** dataset [55].
15. The **Ionosphere** dataset, used for the classification of radar signals from the ionosphere, was introduced in [56,57].
16. **Liverdisorder** dataset [58], which is a medical dataset.
17. **Lymography** dataset[59].
18. **Mammographic** dataset [60], which is a medical dataset regarding the detection of breast tumors.
19. The **Optdigits** dataset, used for the recognition of handwritten digits from optical scans.
20. **Parkinsons** dataset, which is a medical dataset regarding the Parkinson's Disease[61].
21. **Phoneme**, a dataset related to sound.
22. **Pima** dataset, which is a medical dataset used widely [62].
23. **PirVision** dataset [63], that contains occupancy detection data.
24. **Popfailures** dataset [64], used in various experiments regarding climate.
25. **Spiral** dataset, which was produced artificially and contains two classes.
26. **Regions2** dataset [65].
27. **Saheart** dataset [66], a medical dataset used to detect the the presence of heart diseases.
28. **Satimage** dataset, related to satellite images.
29. **Segment** dataset [67], which is an image processing dataset.
30. The **Sonar** dataset [68].
31. **Spambase** dataset, related to the detection of spam emails [69].
32. **Statheart**, which is medical dataset on heart diseases.
33. The **Student** dataset, containing data collected from various school-related studies, was originally presented in [70].
34. **Transfusion** dataset [71].
35. The **WDBC** dataset, a medical dataset for breast tumor diagnosis, was originally presented in [72].
36. The **Wine** dataset, used for wine quality classification, was originally presented in [73,74].
37. **Eeg** dataset, which is an EEG dataset [75] . From this dataset the following cases were used: Z_F_S, ZONF_S and ZO_NF_S.
38. The **Zoo** dataset, used for animal classification, was originally presented in [76].

3.2. Experimental settings

The software used in this work was entirely coded in ANSI C++ with the support of OPTIMUS environment, downloaded from <https://github.com/itsoulos/OPTIMUS/> (accessed on 24 June 2026)[77]. All the experiments were conducted on an AMD Ryzen 5950X with 128GB of RAM and the used operating system was Debian Linux. During the experimental evaluation, a 10-fold cross-validation procedure was applied, in which each dataset was divided into ten subsets (folds). In each validation round, 90% of the instances were used for training, whereas the remaining 10% were reserved for testing. To ensure statistical reliability, each experimental configuration was independently executed 30 times, with a different initialization seed assigned to the random number generator for each run. The

random numbers required during the experiments were generated using the 'drand48()' function provided by the C standard library. The average classification error is defined as:

$$E_C(N(\vec{x}, \vec{w})) = 100 \times \frac{\sum_{i=1}^N (\text{class}(N(\vec{x}_i, \vec{w})) \neq y_i)}{M} \quad (19)$$

Where the set T represents the associated test set, with $T = (x_i, y_i)$, $i = 1, \dots, M$. The settings used in the conducted experiments are shown in Table 1.

Table 1. Experimental settings.

PARAMETER	MEANING	VALUE
N_c	Number of particles/chromosomes	500
N_g	Maximum number of generations	500
k	Number of weights	10
NM	Propagation method	NtoN
K	Number of subpopulations	10
NR	Propagation rate	10
NP	Volumetric quantity of migrated solutions	1
TS	Global termination strategy	ANY

Moreover, the following notation is used in the tables presenting the experimental results:

1. The column DATASET indicates the name of the used dataset.
2. The column BFGS is used for the application of the BFGS optimization procedure [78] to train an artificial neural network [79,80] with 10 processing nodes.
3. The column NEAT [81] represents the results obtained by the NEAT (NeuroEvolution of Augmenting Topologies) method.
4. The column RBF denotes the usage of a Radial Basis Function (RBF) RBF network [82,83] with 10 processing nodes.
5. The column GEN denotes the incorporation of a genetic algorithm [84] with the same parameters as in the proposed method. This algorithm was used to train an artificial neural network with 10 processing nodes.
6. The column OPSO stands for the results obtained by the incorporation of the proposed method.
7. The row labeled AVERAGE presents the mean classification error obtained across the entire collection of datasets.

3.3. Comparison with other methods

Table 2 reports the per-dataset classification error achieved by four baseline machine-learning models (BFGS, NEAT, RBF, GEN) and by the proposed OPSO algorithm under three thread/population configurations ($T = 1$, $T = 2$, $T = 5$), evaluated across 40 benchmark datasets.

Table 2. Comparison with other machine learning methods on the series of classification datasets.

DATASET	BFGS	NEAT	RBF	GEN	OPSO ($T = 1$)	OPSO ($T = 2$)	OPSO ($T = 5$)
APPENDICITIS	18.00%	17.20%	12.23%	18.10%	22.30%	23.30%	22.80%
ALCOHOL	41.50%	66.80%	49.38%	39.57%	17.66%	20.21%	22.13%
AUSTRALIAN	38.13%	31.98%	34.89%	32.21%	29.41%	27.38%	24.43%
BALANCE	8.64%	23.14%	33.53%	8.97%	7.90%	7.32%	6.94%
CIRCULAR	6.08%	35.18%	5.98%	5.99%	4.20%	4.36%	4.34%
CLEVELAND	77.55%	53.44%	67.10%	51.60%	50.79%	46.14%	45.69%
DERMATOLOGY	52.92%	32.43%	62.34%	30.58%	9.60%	10.08%	9.97%
ECOLI	69.52%	43.44%	59.48%	54.67%	51.73%	51.97%	50.67%
FERT	23.20%	15.37%	15.00%	28.50%	25.70%	24.90%	25.50%
HABERMAN	29.34%	24.04%	25.10%	28.66%	30.13%	29.50%	28.97%
HAYES ROTH	37.33%	50.15%	64.36%	56.18%	32.92%	34.85%	35.00%
HEART	39.44%	39.27%	31.20%	28.34%	27.33%	23.11%	21.11%
HEARTATTACK	46.67%	32.34%	29.00%	29.03%	32.43%	24.10%	19.93%
HOUSEVOTES	7.13%	10.89%	6.13%	6.62%	7.09%	5.91%	6.96%
IONOSPHERE	15.29%	19.67%	16.22%	15.15%	15.14%	15.03%	14.86%
LIVERDISORDER	42.59%	30.67%	30.84%	31.11%	35.71%	32.38%	31.94%
LYMOGRAPHY	35.43%	33.70%	25.50%	28.42%	26.22%	25.50%	24.93%
MAMMOGRAPHIC	17.24%	22.85%	21.38%	19.88%	17.05%	17.02%	17.06%
OPTDIGITS	67.13%	72.56%	71.97%	67.81%	56.78%	48.49%	49.39%
PARKINSONS	27.58%	18.56%	17.41%	18.05%	17.21%	15.58%	16.37%
PHONEME	15.58%	22.34%	23.32%	15.55%	14.93%	15.11%	15.46%
PIMA	35.59%	34.51%	25.78%	32.19%	33.12%	30.47%	27.88%
PIRVISION	16.94%	11.84%	24.58%	14.31%	9.31%	10.54%	9.07%
POPFAILURES	5.24%	7.05%	7.04%	5.94%	7.87%	6.92%	6.83%
REGIONS2	36.28%	33.23%	38.29%	29.39%	30.10%	30.13%	31.11%
SAHEART	37.48%	34.51%	32.19%	34.86%	35.63%	34.41%	33.87%
SATIMAGE	71.73%	76.19%	40.19%	54.54%	47.58%	40.01%	47.47%
SEGMENT	68.97%	66.72%	59.68%	57.72%	22.35%	15.21%	17.74%
SONAR	25.85%	34.10%	27.90%	22.40%	23.10%	22.10%	19.80%
SPAMBASE	18.16%	35.77%	29.35%	6.37%	6.50%	6.59%	6.19%
SPIRAL	47.99%	48.66%	44.87%	48.66%	44.81%	43.01%	41.91%
STATHEART	39.65%	44.36%	31.36%	27.25%	28.04%	23.89%	22.22%
STUDENT	7.14%	10.20%	5.49%	5.61%	6.00%	5.68%	5.68%
TRANSFUSION	25.84%	24.87%	26.41%	24.87%	24.35%	24.92%	24.45%
WDBC	29.91%	12.88%	7.27%	8.56%	8.13%	5.80%	4.95%
WINE	59.71%	25.43%	31.41%	19.20%	22.30%	15.77%	11.82%
Z_F_S	39.37%	38.41%	13.16%	10.73%	19.84%	14.17%	9.00%
ZO_NF_S	43.04%	43.75%	9.02%	21.54%	21.98%	12.44%	10.08%
ZONF_S	15.62%	5.44%	4.03%	4.36%	3.66%	3.02%	2.68%
ZOO	10.70%	20.27%	21.93%	9.50%	4.80%	3.40%	3.90%
AVERAGE	33.79%	32.61%	29.56%	26.32%	23.29%	21.27%	20.78%

A closer inspection of the per-dataset results reveals that the advantage of OPSO over the baseline models is far from uniform, but instead concentrated in a specific subset of datasets where the baseline learners struggle markedly. On datasets such as SEGMENT, DERMATOLOGY, ALCOHOL, Z_F_S, and ZO_NF_S, the baseline models yield error rates ranging from roughly 0.31 to 0.69, whereas all three OPSO variants converge to substantially lower values, often below 0.20 and in several cases below 0.10. This pattern suggests that OPSO is particularly effective at navigating the more irregular or multimodal error landscapes that appear to defeat gradient-based or single-pass heuristics such as BFGS and NEAT, rather than offering a uniform, dataset-independent improvement. Examining the behavior across the three OPSO configurations ($T = 1, 2, 5$) shows that increasing the number of threads tends to produce a progressive, though not strictly monotonic, refinement of the solution. Datasets such as WINE, Z_F_S, and ZO_NF_S display a clear, consistent decrease in error from $T = 1$ to $T = 5$, indicating that additional parallel search threads continue to yield tangible benefits even after the first configuration has already improved substantially over the baselines. In a smaller number of cases, such as FERT and SEGMENT, the lowest error is instead observed at $T = 2$ rather than $T = 5$, pointing to occasional diminishing or slightly non-monotonic returns once the population

has already reached a favorable region of the search space. Not every dataset conforms to this favorable pattern, however. On HABERMAN, POPFAILURES, LIVERDISORDER, and TRANSFUSION, the OPSO variants perform on par with or even slightly worse than the best baseline model, with differences generally confined to the second or third decimal place. These are, notably, datasets on which the baseline models themselves already achieve comparatively low and closely clustered error rates, suggesting that once a classification problem is "easy" or nearly linearly separable, the additional search capacity offered by OPSO yields little to no further gain. At the other extreme, datasets such as CLEVELAND, SATIMAGE, ECOLI, and OPTDIGITS remain uniformly difficult for every method tested, with error rates staying above 0.40 regardless of the algorithm used, an indication that the difficulty of these multi-class problems lies in the intrinsic complexity of the data itself rather than in any shortcoming specific to a particular optimization strategy.

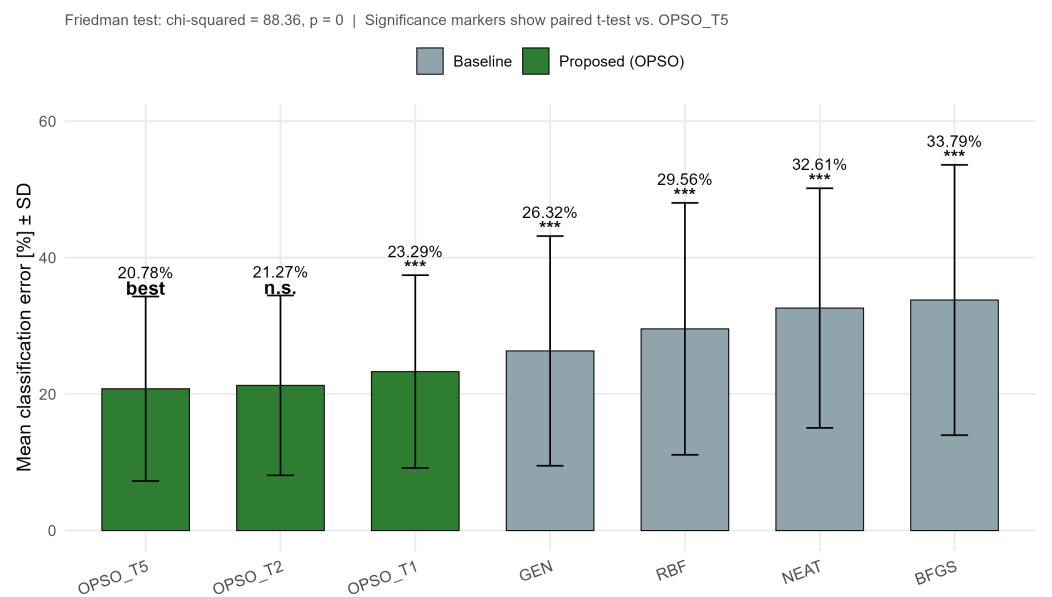


Figure 2. Mean classification error of OPSO ($T = 1, 2, 5$) versus baseline machine-learning models.

Table 3. Statistical comparison of classification error, average ranking, and significance tests (Wilcoxon and paired t-test) for the six machine learning models relative to proposed method as the reference model.

Model	Mean Error	SD Error	Min Error	Max Error	Avg Rank	Mean Diff vs OPSO (T5)	Wilcoxon p	Paired p	Significance	
OPSO T5	Proposed (OPSO)	20.78	13.53	2.68	50.67	2.16	0			Reference
OPSO T2	Proposed (OPSO)	21.27	13.18	3.02	51.97	2.67	-0.49	0.0351	0.1351	n.s.
OPSO T1	Proposed (OPSO)	23.29	14.13	3.66	56.78	3.58	-2.52	0.0001	0.0002	***
GEN	Baseline	26.32	16.84	4.36	67.81	4.08	-5.55	0	0.0001	***
RBF	Baseline	29.56	18.46	4.03	71.97	4.44	-8.78	0.0002	0.0002	***
NEAT	Baseline	32.61	17.56	5.44	76.19	5.38	-11.83	0	0	***
BFGS	Baseline	33.79	19.81	5.24	77.55	5.7	-13.01	0	0	***

Friedman test: chi-squared = 88.36, df = 6, p-value = 0
Nemenyi critical difference (alpha = 0.05): 1.425
Reference model for pairwise tests: OPSO_T5 (auto-selected as the best-performing OPSO variant)
Significance vs. reference (paired t-test): *** p<0.001, ** p<0.01, * p<0.05, n.s. = not significant

A statistical comparison of the OPSO against the other machine learning methods is depicted in Figure 2 and in Table 3. Across the 40 benchmark datasets, the Friedman test revealed statistically significant differences among the seven models under

comparison ($\chi^2 = 88.36$, $df = 6$, $p < 0.001$), with a Nemenyi critical difference of 1.425. The OPSO variant with $T = 5$ threads achieved the lowest mean classification error ($20.78\% \pm 13.53$) and the best average rank (2.16), closely followed by OPSO ($T = 2$) ($21.27\% \pm 13.18$), the difference between the two not reaching statistical significance (paired t-test, $p = 0.135$). In contrast, OPSO ($T = 1$) (23.29%) and all four baseline models, GEN (26.32%), RBF (29.56%), NEAT (32.61%), and BFGS (33.79%), differed significantly from the best-performing variant ($p < 0.001$ in every case), providing strong evidence for the superiority of the proposed OPSO algorithm, particularly under the $T = 2$ and $T = 5$ configurations, over conventional approaches.

3.4. Experiments with the propagation method

Table 4 reports the per-dataset classification error obtained by the proposed OPSO algorithm under four alternative solution-propagation strategies (NtoN, 1to1, 1toN, Nto1), evaluated across the same 40 benchmark datasets.

Table 4. Experiments using the proposed method and a series of propagation techniques. The number of processing threads was set to $T = 5$.

DATASET	NtoN	1to1	1toN	Nto1
APPENDICITIS	22.80%	23.90%	23.30%	23.60%
ALCOHOL	22.13%	17.78%	20.85%	19.79%
AUSTRALIAN	24.43%	24.29%	25.19%	22.32%
BALANCE	6.94%	7.71%	8.08%	7.26%
CIRCULAR	4.34%	4.30%	4.25%	4.35%
CLEVELAND	45.69%	45.66%	47.59%	47.24%
DERMATOLOGY	9.97%	10.06%	10.40%	9.83%
ECOLI	50.67%	51.76%	50.33%	51.58%
FERT	25.50%	24.60%	25.00%	25.40%
HABERMAN	28.97%	29.27%	28.70%	29.53%
HAYES ROTH	35.00%	35.00%	37.31%	35.54%
HEART	21.11%	21.41%	20.18%	20.45%
HEARTATTACK	19.93%	21.53%	20.64%	21.13%
HOUSEVOTES	6.96%	6.48%	7.48%	7.61%
IONOSPHERE	14.86%	15.17%	15.37%	14.89%
LIVERDISORDER	31.94%	33.21%	32.47%	33.50%
LYMOGRAPHY	24.93%	26.07%	27.14%	26.21%
MAMMOGRAPHIC	17.06%	17.12%	17.19%	17.13%
OPTDIGITS	49.39%	49.04%	46.47%	53.38%
PARKINSONS	16.37%	15.95%	15.16%	15.90%
PHONEME	15.46%	14.91%	14.93%	14.17%
PIMA	27.88%	26.59%	26.92%	26.87%
PIRVISION	9.07%	10.52%	7.93%	9.15%
POPFAILURES	6.83%	6.80%	6.61%	7.04%
REGIONS2	31.11%	30.73%	29.40%	30.24%
SAHEART	33.87%	33.65%	33.96%	33.15%
SATIMAGE	47.47%	52.18%	53.19%	54.45%
SEGMENT	17.74%	20.68%	19.07%	18.94%
SONAR	19.80%	22.60%	21.15%	21.90%
SPAMBASE	6.19%	6.52%	6.33%	6.09%
SPIRAL	41.91%	42.48%	42.22%	41.72%
STATHEART	22.22%	21.89%	23.11%	20.37%
STUDENT	5.68%	6.23%	5.90%	6.18%
TRANSFUSION	24.45%	24.42%	24.37%	24.29%
WDBC	4.95%	5.30%	6.02%	5.73%
WINE	11.82%	14.24%	13.30%	14.65%
Z_F_S	9.00%	14.40%	10.63%	13.20%
ZO_NF_S	10.08%	10.64%	8.12%	10.16%
ZONF_S	2.68%	3.22%	2.90%	3.56%
ZOO	3.90%	4.50%	4.10%	5.50%
AVERAGE	20.78%	21.32%	21.08%	21.35%

The most immediately striking behavior in this table is the remarkable consistency of the four propagation strategies on the majority of datasets: for cases such as BALANCE, CIRCULAR, MAMMOGRAPHIC, TRANSFUSION, and PHONEME, the four strategies produce error values that agree to within one or two hundredths, and in some instances within a few thousandths. This tight clustering indicates that, for a large share of the benchmark problems, the specific topology through which good solutions are shared among islands has essentially no bearing on the final outcome, implying that the search process converges to a comparable solution regardless of how information circulates once the population size and number of iterations are held fixed.

A markedly different behavior emerges, however, on a smaller set of more demanding datasets. SATIMAGE shows the widest spread among all strategies (from 0.3717 to a high of 0.5445), and comparable, if less extreme, spreads appear for SEGMENT, WINE, Z_F_S, and ZO_NF_S. On most of these harder cases, the full-connectivity strategy (NtoN) tends to sit at or near the lowest error observed, consistent with the intuition that richer, more thorough mixing of good solutions across islands becomes increasingly valuable as the underlying optimization landscape grows more rugged or deceptive, while restricted propagation schemes may leave promising regions of the search space insufficiently exploited.

At the same time, no single propagation strategy dominates uniformly across the remaining, moderately difficult datasets: Nto1 achieves the lowest error on AUSTRALIAN, PHONEME, and STATHEART, 1to1 is best on HOUSEVOTES, and 1toN performs best on PARKINSONS and HEART, with the margins separating the four strategies in each of these cases remaining small. This idiosyncratic distribution of "wins" across strategies, rather than a consistent ranking, suggests that for datasets of intermediate difficulty the choice of propagation topology interacts with dataset-specific characteristics in ways that are not easily predictable from the properties of the strategy alone, reinforcing the impression that propagation topology becomes a decisive factor mainly at the extremes of problem difficulty rather than across the board.

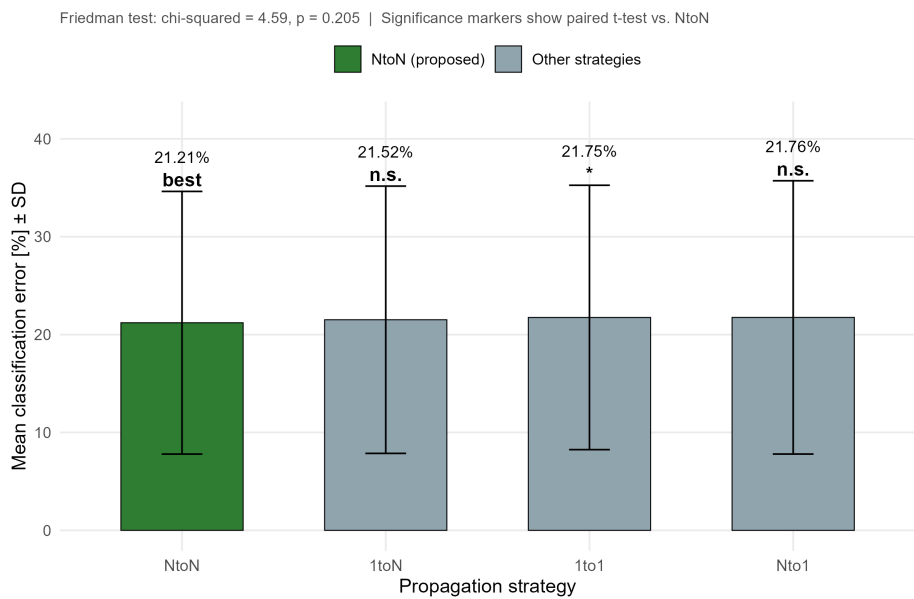


Figure 3. Mean classification error of OPSO under four solution-propagation strategies.

Table 5. Statistical comparison of the four propagation strategies relative to NtoN.

Propagation	Mean Error	SD Error	Min Error	Max Error	Avg Rank	Mean Diff vs NtoN	Wilcoxon p	Paired p	Significance
NtoN	21.21	13.42	2.68	50.67	2.14	0			Reference
1toN	21.52	13.65	2.9	53.19	2.51	-0.31	0.1781	0.1931	n.s.
1to1	21.75	13.51	3.22	52.18	2.73	-0.54	0.0241	0.0411	*
Nto1	21.76	13.96	3.56	54.45	2.62	-0.55	0.0874	0.0571	n.s.

Friedman test: chi-squared = 4.589, df = 3, p-value = 0.2045
Nemenyi critical difference (alpha = 0.05): 0.751
Reference strategy for pairwise tests: NtoN (full propagation)
Significance vs. NtoN (paired t-test): *** p<0.001, ** p<0.01, * p<0.05, n.s. = not significant

A statistical comparison for the propagation strategy is depicted in Figure 3 and in Table 5. Unlike the two preceding comparisons, the Friedman test did not detect a statistically significant overall difference among the four propagation strategies ($\chi^2 = 4.589$, $df = 3$, $p = 0.205$, $n = 39$, with the summary average row excluded). The full-propagation strategy (NtoN) attained the marginally lowest mean error ($21.21\% \pm 13.42$) and the best average rank (2.14), neither 1toN (21.52% , $p = 0.193$) nor Nto1 (21.76% , $p = 0.057$) differed significantly from it, while 1to1 (21.75%) showed only a nominally significant difference ($p = 0.041$) that does not survive correction for multiple comparisons. Taken together, these results indicate that the topology of solution propagation exerts a comparatively limited influence on OPSO's performance relative to factors such as algorithmic variant or population size, which were examined in the preceding analyses.

3.5. Experiments with the number of particles

Table 6 reports the per-dataset classification error obtained by the proposed OPSO algorithm under four different population sizes ($N_c = 200, 500, 1000, 2000$), evaluated across the same 40 benchmark datasets.

Table 6. Experiments using the proposed method and a series of values for the number of particle N_c . The propagation method was set to $NtoN$ and the number of processing threads was set to $T = 5$.

DATASET	$N_c = 200$	$N_c = 500$	$N_c = 1000$	$N_c = 2000$
APPENDICITIS	23.70%	22.80%	21.60%	21.40%
ALCOHOL	20.42%	22.13%	18.09%	14.89%
AUSTRALIAN	23.21%	24.43%	24.78%	23.13%
BALANCE	7.19%	6.94%	7.48%	7.36%
CIRCULAR	4.35%	4.34%	4.52%	4.56%
CLEVELAND	45.41%	45.69%	45.79%	45.72%
DERMATOLOGY	13.29%	9.97%	9.92%	10.03%
ECOLI	53.70%	50.67%	49.27%	50.76%
FERT	24.30%	25.50%	27.60%	26.50%
HABERMAN	28.43%	28.97%	29.37%	28.97%
HAYES ROTH	36.00%	35.00%	36.23%	37.69%
HEART	21.14%	21.11%	19.89%	17.78%
HEARTATTACK	21.63%	19.93%	19.47%	19.33%
HOUSEVOTES	6.61%	6.96%	6.70%	7.30%
IONOSPHERE	14.71%	14.86%	15.72%	13.94%
LIVERDISORDER	34.06%	31.94%	32.47%	32.47%
LYMOGRAPHY	25.86%	24.93%	24.36%	25.36%
MAMMOGRAPHIC	16.78%	17.06%	17.29%	17.26%
OPTDIGITS	44.06%	49.39%	44.43%	47.46%
PARKINSONS	15.79%	16.37%	16.26%	15.05%
PHONEME	15.06%	15.46%	14.59%	15.17%
PIMA	29.03%	27.88%	26.62%	27.04%
PIRVISION	11.11%	9.07%	7.72%	8.69%
POPFAILURES	6.78%	6.83%	7.04%	6.82%
REGIONS2	29.15%	31.11%	29.18%	27.79%
SAHEART	33.91%	33.87%	33.00%	32.67%
SATIMAGE	58.17%	47.47%	52.88%	37.17%
SEGMENT	19.83%	17.74%	16.46%	17.21%
SONAR	21.35%	19.80%	21.65%	20.15%
SPAMBASE	6.35%	6.19%	5.96%	6.33%
SPIRAL	42.04%	41.91%	42.41%	41.29%
STATHEART	23.96%	22.22%	21.96%	19.41%
STUDENT	6.50%	5.68%	5.45%	5.55%
TRANSFUSION	24.57%	24.45%	24.62%	24.16%
WDBC	5.61%	4.95%	5.39%	4.43%
WINE	14.12%	11.82%	10.59%	11.41%
Z_F_S	13.47%	9.00%	10.57%	8.73%
ZO_NF_S	8.68%	10.08%	6.38%	6.34%
ZONF_S	3.16%	2.68%	2.86%	2.96%
ZOO	4.50%	3.90%	3.90%	4.30%
AVERAGE	21.45%	20.78%	20.51%	19.86%

For a substantial portion of the datasets, increasing the population size from $N_c = 200$ to $N_c = 2000$ produces a visible, and in several cases fairly consistent, downward trend in error. HEART (from 0.2114 to 0.1778), STATHEART (from 0.2396 to 0.1941), HEARTATTACK (from 0.2163 to 0.1933), and WDBC (from 0.0561 to 0.0443) all display this steady improvement, indicating that a larger population provides these problems with a genuinely richer sampling of the search space that translates into more accurate final solutions rather than merely additional computational overhead.

A second, equally prominent behavior is the near-total insensitivity of certain datasets to population size. On MAMMOGRAPHIC, TRANSFUSION, HABERMAN, SAHEART, CLEVELAND, BALANCE, and CIRCULAR, the error stays within a narrow band of no

more than roughly one hundredth across all four population sizes, suggesting that for these particular problems the search already converges to essentially the same solution with as few as 200 individuals, and that the additional computational cost associated with larger populations offers no discernible practical benefit for this class of dataset.

Finally, a smaller subset of datasets displays behavior that runs counter to the general trend, either through pronounced instability or through outright deterioration with a larger population. SATIMAGE stands out with the widest fluctuation observed anywhere in the table (ranging non-monotonically from 0.5817 down to 0.3717), while FERT (rising from 0.243 to 0.276 before partially recovering) and HAYES ROTH (lowest at $N_c = 500$, then rising steadily to 0.3769 at $N_c = 2000$) actually perform worse, on balance, with a larger population. This behavior indicates that the relationship between population size and solution quality is not universally monotonic, and that for certain datasets, likely those with noisier or more deceptive fitness landscapes, an overly large population may dilute selective pressure or slow convergence within a fixed number of iterations, underscoring that population size should ideally be tuned per problem rather than fixed at a single "optimal" value for all datasets.

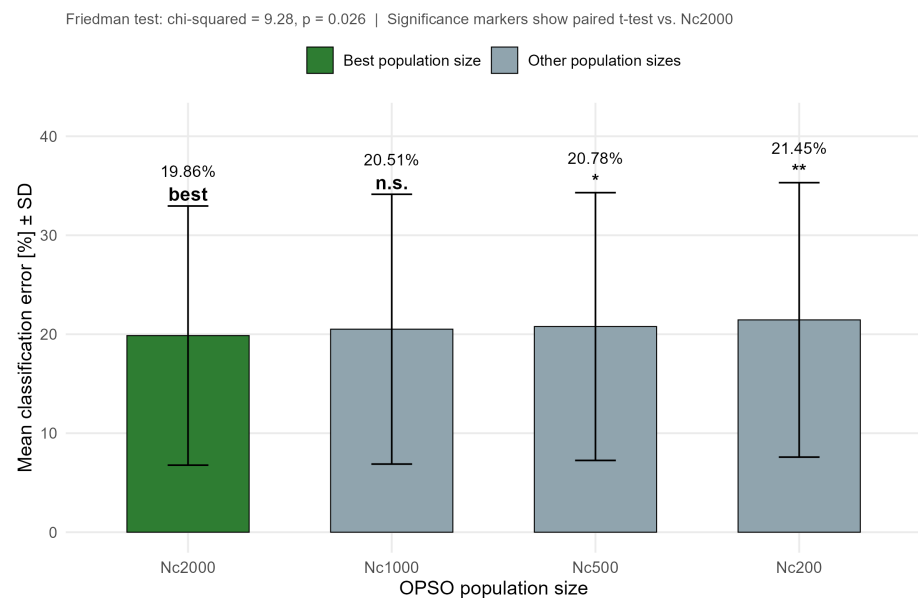


Figure 4. Mean classification error of OPSO under four population sizes.

Table 7. Statistical comparison of the four population sizes relative to $N_c = 2000$.

N_c	Mean Error	SD Error	Min Error	Max Error	Avg Rank	Mean Diff vs $N_c = 2000$	Wilcoxon p	Paired p	Significance
2000	19.86	13.09	2.96	50.76	2.08	0			Reference
1000	20.51	13.63	2.86	52.88	2.48	-0.65	0.1428	0.1401	n.s.
500	20.78	13.53	2.68	50.67	2.5	-0.91	0.0087	0.0132	*
200	21.45	13.86	3.16	58.17	2.95	-1.59	0.0005	0.0089	**

Friedman test: chi-squared = 9.28, df = 3, p-value = 0.0258

Nemenyi critical difference (alpha = 0.05): 0.742

Reference model for pairwise tests: $N_c=2000$ (auto-selected as the best-performing population size)

Significance vs. reference (paired t-test): *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, n.s. = not significant

The statistical comparison for the obtained results is presented in Figure 4 and in Table 7. The Friedman test indicated an overall statistically significant effect of population size on algorithmic performance ($\chi^2 = 9.28$, df = 3, $p = 0.026$, $n = 40$), with a Nemenyi critical

difference of 0.742. A population size of $N_c = 2000$ yielded the lowest mean error (19.86% $\pm$ 13.09) and the best average rank (2.08), while $N_c = 1000$ did not differ significantly from this reference (20.51%, $p = 0.140$). Smaller populations, however, performed significantly worse: $N_c = 500$ (20.78%, $p = 0.013$) and, more markedly, $N_c = 200$ (21.45%, $p = 0.009$). The complete, Holm-corrected pairwise comparisons (provided as Supplementary Material) confirmed that this effect is driven primarily by the smallest population size ($N_c = 200$ vs. $N_c = 1000$: paired t-test $p = 0.011$ (Holm-corrected); vs. $N_c = 2000$: $p = 0.045$ (Holm-corrected)), whereas the differences among $N_c = 500$, $N_c = 1000$, and $N_c = 2000$ did not remain significant after correction, suggesting diminishing returns beyond a population size of approximately 500–1000 individuals.

4. Conclusions

This paper introduced OPSO, a parallel, multi-population optimization algorithm for classification tasks, and evaluated it systematically against four established machine-learning models across 40 benchmark datasets drawn from the UCI and KEEL repositories. Across three thread/population configurations, OPSO consistently and significantly outperformed all four baseline models, with the $T = 5$ configuration achieving the lowest overall mean error and the $T = 2$ configuration performing statistically indistinguishably from it. Importantly, this advantage was not uniform across datasets: it was most pronounced on problems exhibiting irregular or multimodal error surfaces, precisely the class of problems where gradient-based or single-pass heuristics are known to struggle, while on datasets that were already comparatively easy for the baseline models, OPSO offered little to no additional benefit. This pattern suggests that the proposed method's principal strength lies not in universally improving classification accuracy, but in providing a more capable search mechanism specifically for the harder, more deceptive regions of the problem space.

The two supplementary experiments clarified which internal design choices of OPSO meaningfully affect its performance and which do not. Increasing the population size produced a statistically significant improvement, though with clearly diminishing returns beyond roughly 500–1000 particles, and with a subset of datasets showing negligible or even mildly non-monotonic sensitivity to this parameter, indicating that population size, while important, should be regarded as a tunable, problem-dependent setting rather than a fixed universal choice. By contrast, the solution-propagation strategy used to share promising candidates among sub-populations had no statistically significant overall effect, with the full-connectivity strategy (NtoN) offering only a marginal, largely non-significant advantage over more restricted topologies. Taken together, these results position OPSO as a robust and practically competitive alternative to conventional classifier-training approaches, whose effectiveness is driven more by the number and diversity of its parallel search threads and the richness of its populations than by the specific manner in which information is exchanged among them.

The present study, while comprehensive in its comparison across baseline models and internal configurations, leaves several directions open for further investigation. First, the observation that OPSO's advantage is concentrated in datasets with irregular or multimodal error landscapes warrants a more systematic characterization of dataset difficulty, for instance, through measures of class overlap, intrinsic dimensionality, or landscape ruggedness, so that the conditions under which OPSO is expected to excel can be predicted a priori rather than identified only after the fact. Second, the non-monotonic or even adverse effect of increasing population size observed on a small number of datasets (e.g., FERT, HAYES ROTH) suggests that an adaptive or self-tuning population size, adjusted dynamically during the search based on convergence behavior, may outperform any single fixed value and merits dedicated investigation. Third, although the propagation strategy was found

to have limited overall impact, its comparatively larger effect on the hardest benchmark problems (e.g., SATIMAGE, SEGMENT) indicates that hybrid or adaptive propagation schemes, which shift between restricted and fully connected topologies depending on the observed diversity or stagnation of the population, could yield further improvements specifically on difficult problems, and should be explored in future work. Finally, extending the evaluation beyond the 40 datasets considered here to larger-scale, higher-dimensional, or streaming classification problems, and benchmarking OPSO against a broader range of modern evolutionary and swarm-based optimizers, would help establish the generality of the conclusions reached in this study and clarify the computational trade-offs involved in deploying OPSO at scale.

Author Contributions: Conceptualization, K.B. and I.G.T.; methodology, K.B.; software, K.B.; validation, V.C., G.K. and I.G.T.; formal analysis, V.C.; investigation, I.G.T.; resources, G.K.; data curation, K.B.; writing—original draft preparation, K.B.; writing—review and editing, I.G.T.; visualization, V.C.; supervision, I.G.T.; project administration, I.G.T.; funding acquisition, G.K.. All authors have read and agreed to the published version of the manuscript.

Funding: This research has been financed by the European Union: Next Generation EU through the Program Greece 2.0 National Recovery and Resilience Plan, under the call RESEARCH—CREATE—INNOVATE, project name “iCREW: Intelligent small craft simulator for advanced crew training using Virtual Reality techniques” (project code:TAEDK-06195).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhang, A.; Lipton, Z.C.; Li, M.; Smola, A.J. Dive into Deep Learning. *arXiv* **2023**, arXiv:2106.11342v5, <https://doi.org/10.48550/arXiv.2106.11342>.
2. Kaveh, M.; Mesgari, M.S. Application of Meta-Heuristic Algorithms for Training Neural Networks and Deep Learning Architectures: A Comprehensive Review. *Neural Process. Lett.* **2022**, *54*, 1–104, <https://doi.org/10.1007/s11063-022-11055-6>.
3. Charilogis, V.; Kyrou, G.; Tsoulos, I.G.; Gianni, A.M. An Innovative Hybrid Approach Producing Trial Solutions for Global Optimization. *Appl. Sci.* **2024**, *14*, 10567. <https://doi.org/10.3390/app142210567>.
4. Dauphin, Y.N.; Pascanu, R.; Gulcehre, C.; Cho, K.; Ganguli, S.; Bengio, Y. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. *Adv. Neural Inf. Process. Syst.* **2014**, *27*, 2933–2941.
5. Rebello, C.M.; Martins, M.A.F.; Loureiro, J.M.; Rodrigues, A.E.; Ribeiro, A.M.; Nogueira, I.B.R. From an Optimal Point to an Optimal Region: A Novel Methodology for Optimization of Multimodal Constrained Problems and a Novel Constrained Sliding Particle Swarm Optimization Strategy. *Mathematics* **2021**, *9*, 1808. <https://doi.org/10.3390/math9151808>
6. Metropolis, N.; Rosenbluth, A.W.; Rosenbluth, M.N.; Teller, A.H.; Teller, E. Equation of State Calculations by Fast Computing Machines. *J. Chem. Phys.* **1953**, *21*, 1087–1092, <https://doi.org/10.1063/1.1699114>.
7. Carleo, G.; Troyer, M. Solving the Quantum Many-Body Problem with Artificial Neural Networks. *Science* **2017**, *355*, 602–606, <https://doi.org/10.1126/science.aag2302>.
8. Xia, X.; Zhang, Y.; Zeng, X.; Zhang, X.; Zheng, C.; Su, Y. Artificial Intelligence in Molecular Optimization: Current Paradigms and Future Frontiers. *Int. J. Mol. Sci.* **2025**, *26*, 4878. <https://doi.org/10.3390/ijms26104878>.
9. Bueno-Ferrer, Á.; De Pablo Valenciano, J.; De Burgos Jiménez, J. Unveiling the Potential of Metaheuristics in Transportation: A Path Towards Efficiency, Optimization, and Intelligent Management. *Infrastructures* **2025**, *10*, 4. <https://doi.org/10.3390/infrastructures10010004>.
10. Andrès, E.; Escobar, C.; Doi, K. Machine Learning and Artificial Intelligence in Clinical Medicine—Trends, Impact, and Future Directions. *J. Clin. Med.* **2025**, *14*, 8137. <https://doi.org/10.3390/jcm14228137>.
11. Shehata, M.; Elhosseini, M. Advances in AI Technology in Healthcare. *Bioengineering* **2025**, *12*, 506. <https://doi.org/10.3390/bioengineering12050506>.

12. Fernández, J.; G.-Tóth, B. Interval Tools in Branch-and-Bound Methods for Global Optimization. In *The Palgrave Handbook of Operations Research*; Salhi, S., Boylan, J., Eds.; Palgrave Macmillan: Cham, Switzerland, **2022**; pp. 263–296, https://doi.org/10.1007/978-3-030-96935-6_8. 749–751
13. Gillner, L. Efficient acceleration strategies for interval branch-and-bound type methods. *Int. J. Approx. Reason.* **2026**, *198*, 109767, <https://doi.org/10.1016/j.ijar.2026.109767>. 752–753
14. Rumelhart, D.E.; Hinton, G.E.; Williams, R.J. Learning representations by back-propagating errors. *Nature* **1986**, *323*, 533–536, <https://doi.org/10.1038/323533a0>. 754–755
15. Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*; MIT Press: Cambridge, MA, USA, **2016**. 756
16. Yang, X.-S. *Nature-Inspired Optimization Algorithms*, 2nd ed.; Academic Press: Cambridge, MA, USA, **2020**. 757
17. Price, W.L. A controlled random search procedure for global optimisation. *Comput. J.* **1977**, *20*, 367–370, <https://doi.org/10.1093/comjnl/20.4.367>. 758–759
18. Kirkpatrick, S.; Gelatt, C.D., Jr.; Vecchi, M.P. Optimization by Simulated Annealing. *Science* **1983**, *220*, 671–680, <https://doi.org/10.1126/science.220.4598.671>. 760–761
19. Kuo, C.L.; Kuruoglu, E.E.; Chan, W.K.V. Neural Network Structure Optimization by Simulated Annealing. *Entropy* **2022**, *24*, 348, <https://doi.org/10.3390/e24030348>. 762–763
20. Storn, R.; Price, K. Differential Evolution—A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. *J. Glob. Optim.* **1997**, *11*, 341–359, <https://doi.org/10.1023/A:1008202821328>. 764–765
21. Gad, A.G. Particle Swarm Optimization Algorithm and Its Applications: A Systematic Review. *Arch. Comput. Methods Eng.* **2022**, *29*, 2531–2561, <https://doi.org/10.1007/s11831-021-09694-4>. 766–767
22. Poli, R.; Kennedy, J.; Blackwell, T. Particle swarm optimization. *Swarm Intell.* **2007**, *1*, 33–57, <https://doi.org/10.1007/s11721-007-0002-0>. 768–769
23. Dorigo, M.; Birattari, M.; Stützle, T. Ant colony optimization. *IEEE Comput. Intell. Mag.* **2006**, *1*, 28–39, <https://doi.org/10.1109/MCI.2006.329691>. 770–771
24. Holland, J.H. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*; MIT Press: Cambridge, MA, USA, **1992**. 772–773
25. Essaid, M.; Idoumghar, L.; Lepagnot, J.; Bréviliers, M. GPU parallelization strategies for metaheuristics: a survey. *Int. J. Parallel Emergent Distrib. Syst.* **2019**, *34*, 497–522, <https://doi.org/10.1080/17445760.2018.1428969>. 774–775
26. Cano, A.; García-Martínez, C. 100 Million dimensions large-scale global optimization using distributed GPU computing. In *Proceedings of the 2016 IEEE Congress on Evolutionary Computation (CEC)*, **2016**; pp. 3566–3573, <https://doi.org/10.1109/CEC.2016.7744241>. 776–778
27. Houssein, E.H.; Saeed, M.K.; Hu, G.; Ibrahim, I.E. Metaheuristics for Solving Global and Engineering Optimization Problems: Review, Applications, Open Issues and Challenges. *Arch. Comput. Methods Eng.* **2024**, *31*, 4485–4519, <https://doi.org/10.1007/s11831-024-10168-6>. 779–781
28. Sun, Y.; Chu, S.-C.; Hu, P.; Watada, J.; Si, M.; Pan, J.-S. Overview of Parallel Computing for Meta-Heuristic Algorithms. *J. Netw. Intell.* **2022**, *7*, 656–684. 782–783
29. Salgotra, R.; Chaudhary, R.; Verma, P. The Rise of Particle Swarm Optimization: A Systematic Bibliometric & Thematic Review (1995–2025). *Arch. Comput. Methods Eng.* **2026**, <https://doi.org/10.1007/s11831-026-10532-8>. 784–785
30. Rathore, A.; Sharma, H. A Review on Inertia Weight Strategies for Particle Swarm Optimization. In *Proceedings of Sixth International Conference on Soft Computing for Problem Solving*; Deep, K., Ed.; Advances in Intelligent Systems and Computing; Springer: Singapore, **2017**; Vol. 547, pp. 88–98. 786–788
31. Charilogis, V.; Tsoulos, I.G. Toward an Ideal Particle Swarm Optimizer for Multidimensional Functions. *Information* **2022**, *13*, 217. 789
32. de Melo Menezes, B.A.; Kuchen, H.; Buarque de Lima Neto, F. Parallelization of Swarm Intelligence Algorithms: Literature Review. *Int. J. Parallel Program.* **2022**, *50*, 486–514, <https://doi.org/10.1007/s10766-022-00736-3>. 790–791
33. Charilogis, V.; Tsoulos, I.G.; Tzallas, A. An Improved Parallel Particle Swarm Optimization. *SN Comput. Sci.* **2023**, *4*, 766, <https://doi.org/10.1007/s42979-023-02227-9>. 792–793
34. Nocedal, J.; Wright, S.J. *Numerical Optimization*, 2nd ed.; Springer: New York, NY, USA, **2006**. 794
35. Kennedy, J.; Eberhart, R. Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, Perth, WA, Australia, **1995**; Vol. 4, pp. 1942–1948, <https://doi.org/10.1109/ICNN.1995.488968>. 795–796
36. Shi, Y.; Eberhart, R. A modified particle swarm optimizer. In *Proceedings of the 1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence*, Anchorage, AK, USA, **1998**; pp. 69–73, <https://doi.org/10.1109/ICEC.1998.699146>. 797–799
37. OpenMP Architecture Review Board. *OpenMP Application Programming Interface Version 6.0*; OpenMP ARB: Beaverton, OR, USA, **2024**. Available online: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf> (accessed on 21 July 2026). 800–802

38. Kyrrou, G.; Barkas, K.G.; Charilogis, V.; Tsoulos, I.G. Parallel Balanced Grey Wolf Optimizer: A Cooperative Parallel Approach for Large-Scale Optimization Problems. *Foundations* **2026**, *6*, 21. <https://doi.org/10.3390/foundations6020021>. 803
39. da Silveira, L.A.; Soncco-Álvarez, J.L.; de Lima, T.A.; Ayala-Rincón, M. Behavior of Bioinspired Algorithms in Parallel Island Models. In *Proceedings of the 2020 IEEE Congress on Evolutionary Computation (CEC)*, Glasgow, UK, **2020**; pp. 1–8. <https://doi.org/10.1109/CEC48606.2020.9185732>. 804
40. Abadlia, H.; Smairi, N.; Ghedira, K. Particle swarm optimization based on island models. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*; Association for Computing Machinery: New York, NY, USA, **2017**; pp. 101–102. <https://doi.org/10.1145/3067695.3076068>. 805
41. Tsoulos, I.G.; Tzallas, A.; Karvounis, E. Improving the PSO method for global optimization problems. *Evolving Systems* **2021**, *12*, 875–883. <https://doi.org/10.1007/s12530-020-09330-9>. 806
42. Petalas, Y.G.; Parsopoulos, K.E.; Vrahatis, M.N. Memetic particle swarm optimization. *Ann. Oper. Res.* **2007**, *156*, 99–127. <https://doi.org/10.1007/s10479-007-0224-y>. 807
43. M. Kelly, R. Longjohn, K. Nottingham, The UCI Machine Learning Repository, <https://archive.ics.uci.edu>. 808
44. J. Alcalá-Fdez, A. Fernandez, J. Luengo, J. Derrac, S. García, L. Sánchez, F. Herrera. KEEL Data-Mining Software Tool: Data Set Repository, Integration of Algorithms and Experimental Analysis Framework. *Journal of Multiple-Valued Logic and Soft Computing* **17**, pp. 255–287, 2011. 809
45. Weiss, Sholom M. and Kulikowski, Casimir A., *Computer Systems That Learn: Classification and Prediction Methods from Statistics, Neural Nets, Machine Learning, and Expert Systems*, Morgan Kaufmann Publishers Inc, 1991. 810
46. Tzimourta, K.D.; Tsoulos, I.; Bilero, I.T.; Tzallas, A.T.; Tsiouras, M.G.; Giannakeas, N. Direct Assessment of Alcohol Consumption in Mental State Using Brain Computer Interfaces and Grammatical Evolution. *Inventions* **2018**, *3*, 51. 811
47. J.R. Quinlan, Simplifying Decision Trees. *International Journal of Man-Machine Studies* **27**, pp. 221–234, 1987. 812
48. T. Shultz, D. Mareschal, W. Schmidt, Modeling Cognitive Development on Balance Scale Phenomena, *Machine Learning* **16**, pp. 59–88, 1994. 813
49. Z.H. Zhou, Y. Jiang, NeC4.5: neural ensemble based C4.5," in *IEEE Transactions on Knowledge and Data Engineering* **16**, pp. 770–773, 2004. 814
50. R. Setiono, W.K. Leow, FERNN: An Algorithm for Fast Extraction of Rules from Neural Networks, *Applied Intelligence* **12**, pp. 15–25, 2000. 815
51. G. Demiroz, H.A. Govenir, N. Ilter, Learning Differential Diagnosis of Erythematous-Squamous Diseases using Voting Feature Intervals, *Artificial Intelligence in Medicine*. **13**, pp. 147–165, 1998. 816
52. P. Horton, K. Nakai, A Probabilistic Classification System for Predicting the Cellular Localization Sites of Proteins, In: *Proceedings of International Conference on Intelligent Systems for Molecular Biology* **4**, pp. 109–115, 1996. 817
53. B. Hayes-Roth, B., F. Hayes-Roth. Concept learning and the recognition and classification of exemplars. *Journal of Verbal Learning and Verbal Behavior* **16**, pp. 321–338, 1977. 818
54. I. Kononenko, E. Šimec, M. Robnik-Šikonja, Overcoming the Myopia of Inductive Learning Algorithms with RELIEFF, *Applied Intelligence* **7**, pp. 39–55, 1997. 819
55. R.M. French, N. Chater, Using noise to compute error surfaces in connectionist networks: a novel means of reducing catastrophic forgetting, *Neural Comput.* **14**, pp. 1755–1769, 2002. 820
56. J.G. Dy, C.E. Brodley, Feature Selection for Unsupervised Learning, *The Journal of Machine Learning Research* **5**, pp 845–889, 2004. 821
57. S. J. Perantonis, V. Virvilis, Input Feature Extraction for Multilayered Perceptrons Using Supervised Principal Component Analysis, *Neural Processing Letters* **10**, pp 243–252, 1999. 822
58. J. Garcke, M. Griebel, Classification with sparse grids using simplicial basis functions, *Intell. Data Anal.* **6**, pp. 483–502, 2002. 823
59. G. Cestnik, I. Kononenko, I. Bratko, Assistant-86: A Knowledge-Elicitation Tool for Sophisticated Users. In: Bratko, I. and Lavrac, N., Eds., *Progress in Machine Learning*, Sigma Press, Wilmslow, pp. 31–45, 1987. 824
60. M. Elter, R. Schulz-Wendland, T. Wittenberg, The prediction of breast cancer biopsy outcomes using two CAD approaches that both emphasize an intelligible decision process, *Med Phys.* **34**, pp. 4164–72, 2007. 825
61. Little MA, McSharry PE, Hunter EJ, Spielman J, Ramig LO. Suitability of dysphonia measurements for telemonitoring of Parkinson's disease. *IEEE Trans Biomed Eng.* 2009;56(4):1015. doi:10.1109/TBME.2008.2005954 826
62. J.W. Smith, J.E. Everhart, W.C. Dickson, W.C. Knowler, R.S. Johannes, Using the ADAP learning algorithm to forecast the onset of diabetes mellitus, In: *Proceedings of the Symposium on Computer Applications and Medical Care* IEEE Computer Society Press, pp.261–265, 1988. 827
63. Emad-Ud-Din, M., & Wang, Y. (2023). Promoting occupancy detection accuracy using on-device lifelong learning. *IEEE Sensors Journal*, *23*(9), 9595–9606. 828
64. D.D. Lucas, R. Klein, J. Tannahill, D. Ivanova, S. Brandon, D. Domyancic, Y. Zhang, Failure analysis of parameter-induced simulation crashes in climate models, *Geoscientific Model Development* **6**, pp. 1157–1171, 2013. 829

65. Giannakeas, N., Tsipouras, M.G., Tzallas, A.T., Kyriakidi, K., Tsianou, Z.E., Manousou, P., Hall, A., Karvounis, E.C., Tsianos, V., Tsianos, E. A clustering based method for collagen proportional area extraction in liver biopsy images (2015) Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society, EMBS, 2015-November, art. no. 7319047, pp. 3097-3100.
66. T. Hastie, R. Tibshirani, Non-parametric logistic and proportional odds regression, *JRSS-C (Applied Statistics)* **36**, pp. 260–276, 1987.
67. M. Dash, H. Liu, P. Scheuermann, K. L. Tan, Fast hierarchical clustering and its validation, *Data & Knowledge Engineering* **44**, pp. 109–138, 2003.
68. Gorman, R.P.; Sejnowski, T.J. Analysis of Hidden Units in a Layered Network Trained to Classify Sonar Targets. *Neural Netw.* 1988, *1*, 75–89.
69. Cranor, Lorrie F., LaMacchia, Brian A. Spam!, *Communications of the ACM*, 41(8):74-83, 1998.
70. P. Cortez, A. M. Gonçalves Silva, Using data mining to predict secondary school student performance, In Proceedings of 5th FUTURE BUSINESS TECHNOLOGY CONFERENCE (FUBUTEC 2008) (pp. 5–12). EURO-SIS-ETI, 2008.
71. I-Cheng Yeh, King-Jang Yang, Tao-Ming Ting, Knowledge discovery on RFM model using Bernoulli sequence, *Expert Systems with Applications* **36**, pp. 5866-5871, 2009.
72. W.H. Wolberg, O.L. Mangasarian, Multisurface method of pattern separation for medical diagnosis applied to breast cytology, *Proc Natl Acad Sci U S A.* **87**, pp. 9193–9196, 1990.
73. M. Raymer, T.E. Doom, L.A. Kuhn, W.F. Punch, Knowledge discovery in medical and biological datasets using a hybrid Bayes classifier/evolutionary algorithm. *IEEE transactions on systems, man, and cybernetics. Part B, Cybernetics : a publication of the IEEE Systems, Man, and Cybernetics Society*, **33**, pp. 802-813, 2003.
74. P. Zhong, M. Fukushima, Regularized nonsmooth Newton method for multi-class support vector machines, *Optimization Methods and Software* **22**, pp. 225-236, 2007.
75. R.G. Andrzejak, K. Lehnertz, F. Mormann, C. Rieke, P. David, and C. E. Elger, Indications of nonlinear deterministic and finite-dimensional structures in time series of brain electrical activity: Dependence on recording region and brain state, *Phys. Rev. E* **64**, pp. 1-8, 2001.
76. M. Koivisto, K. Sood, Exact Bayesian Structure Discovery in Bayesian Networks, *The Journal of Machine Learning Research* **5**, pp. 549–573, 2004.
77. Tsoulos, I. G., Charillogis, V., Kyrou, G., Stavrou, V. N., & Tzallas, A. (2025). OPTIMUS: A Multidimensional Global Optimization Package. *Journal of Open Source Software*, 10(108), 7584.
78. Yuan, Y. X. (1991). A modified BFGS algorithm for unconstrained optimization. *IMA Journal of Numerical Analysis*, 11(3), 325-332.
79. C. Bishop, *Neural Networks for Pattern Recognition*, Oxford University Press, 1995.
80. G. Cybenko, Approximation by superpositions of a sigmoidal function, *Mathematics of Control Signals and Systems* **2**, pp. 303-314, 1989.
81. K. O. Stanley, R. Miikkulainen, Evolving Neural Networks through Augmenting Topologies, *Evolutionary Computation* **10**, pp. 99-127, 2002.
82. J. Park and I. W. Sandberg, Universal Approximation Using Radial-Basis-Function Networks, *Neural Computation* **3**, pp. 246-257, 1991.
83. G.A. Montazer, D. Giveki, M. Karami, H. Rastegar, Radial basis function neural networks: A review. *Comput. Rev. J* **1**, pp. 52-74, 2018.
84. Reynolds, J., Rezgui, Y., Kwan, A., & Piriou, S. (2018). A zone-level, building energy optimisation combining an artificial neural network, a genetic algorithm, and model predictive control. *Energy*, 151, 729-739.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.